

Hooks

Os [React Hooks](#) foram introduzidos na biblioteca a partir da versão 16.8. São recursos que permitem o gerenciamento de estado, ciclos de vida do componente e outros recursos do React sem precisar escrever componentes em forma de classes.

React fornece alguns hooks como o *useState*, *useEffect*, mas também permite que você [crie seus próprios hooks](#) para reutilizar em diferentes componentes.

Antes da inclusão dos hooks na biblioteca, os componentes eram escritos em forma de classes, utilizando funções como *componentDidMount()*, *componentDidUpdate()*, entre outras.

```
import React, {Component} from "react";

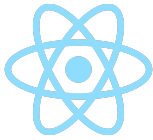
class ComponentExample extends Component {
  render(){
    return(
      <div>Component with class</div>
    )
  }
}
```

Com a utilização dos hooks é possível extrair lógica com estado de um componente de forma que possa ser testada independentemente e reutilizada. É possível reutilizar lógica com estado sem mudar a hierarquia de componentes, tornando fácil o compartilhamento de hooks com vários outros componentes.

Outra vantagem que os hooks oferecem é a possibilidade de dividir um componente em funções menores baseadas em pedaços que são relacionados, em vez de forçar uma divisão baseada nos métodos de ciclo de vida.

Há algumas regras na utilização dos hooks:

- Chame os hooks apenas no *nível mais alto*, pois não devem ser utilizados em loops, condições ou funções aninhadas.
- Apenas chame hooks de *componentes funcionais*, não chame hooks de funções JavaScript comuns.
- Declare seus hooks no topo do componente.



Hooks de Estado

O estado possibilita que um componente “lembre” de uma informação. Por exemplo, em um formulário o estado pode ser usado para armazenar o valor de entrada de um input. Os hooks utilizados para armazenamento de estado são:

- **useState**: variável de estado que pode ser diretamente atualizada. Retorna um array com o valor do estado que será manipulado e uma função que permite alterar o estado para outro valor.

```
const [index, setIndex] = useState(0);
```

- **useReducer**: variável de estado com a lógica de atualização dentro de uma função. É uma forma de implementar um redutor sem utilizar o Redux. Ele retorna um array com o state para o redutor e o dispatch.

```
const [sum, dispatch] = useReducer((state, action) => {  
  return state + action;  
}, 0);
```

Hooks de Performance

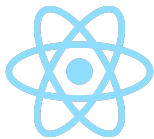
Uma forma de otimizar o desempenho de uma nova renderização é ignorar o trabalho desnecessário. Por exemplo, você pode querer utilizar um cálculo em cache ou não fazer uma nova renderização se alguns dados não tiverem sido alterados desde a renderização anterior.

- **useMemo**: armazena alguns dados em cache. Recebe uma função e um array de argumentos que serão utilizados para comparação. A função será executada junto com a renderização. Retorna o valor memorizado, auxilia na economia de processamento da aplicação.

```
const counterDouble = useMemo(() => counter * 2, [counter]);
```

- **useCallback**: utilizado para memorizar o retorno de chamada fornecido e atualizá-lo quando as dependências fornecidas forem alteradas. Recebe dois parâmetros, uma função callback e um array de dependências, retornando uma versão memorizada da função recebida.

```
const handleClick = useCallback(async () => {  
  await sendMail(address)  
}, [address]);
```



Hooks de Contexto

O React Context API é um gerenciador de estado global que possui o mesmo princípio do Redux, sendo que o Context API é nativo do próprio React. Através da utilização de contexto, um componente filho recebe informações de um componente pai sem a necessidade de passá-las por props.

- [useContext](#): recebe os dados do contexto, similar a criação de uma variável de estado.

```
const theme = useContext(ThemeContext);
```

Hooks de Referência

As referências permitem que um componente mantenha algumas informações que não são usadas para renderização, como um elemento DOM. Ao contrário do estado, a atualização de uma referência não renderiza novamente seu componente.

- [useRef](#): atua como uma função que retorna um objeto ref e recebe um argumento que inicializa a propriedade `.current` desse objeto. Com esse objeto é possível acessar elementos DOM e elementos do próprio React, além de manter uma variável mutável em componentes funcionais.

```
const inputRef = useRef(null);
```

Hooks de Efeito

Os efeitos adicionam a funcionalidade de executar efeitos colaterais através de um componente funcional. Muito utilizado para lidar com chamadas à API, tarefas assíncronas, modificações na DOM, etc.

- [useEffect](#): o primeiro parâmetro que irá receber é uma função de callback, que irá ser disparada após inicialização e atualização do componente. Como segundo parâmetro é passado um array com as variáveis que irão controlar esse hook. Passando um array vazio hook será executado apenas uma vez, na inicialização do componente. É possível também não passar nenhum parâmetro, sendo que nesse caso o hook será executado na inicialização e em todas as atualizações do componente.

```
useEffect(() => {  
  Products.search(product);  
}, [product]);
```

